

FlutterTap

Relatório de Desenvolvimento

Módulo Zygisk (Magisk / KernelSU / APatch) para interceptação de tráfego de apps Flutter

Desenvolvido por Eduardo Lopes

Data: 26 de julho de 2026

1. Resumo executivo

Este projeto porta a lógica do script Frida `flutter+burp.js` — que contorna a verificação de certificado TLS (SSL pinning) do BoringSSL usado pelo engine do Flutter e redireciona o tráfego de rede do app para um proxy configurável — para um **módulo Zyigisk nativo**, compatível simultaneamente com Magisk, KernelSU (via Zyigisk/Zyigisk Next) e APatch. Diferente do script original, que depende de uma sessão manual do Frida (via USB, com `frida-server` rodando), o módulo funciona de forma persistente e automática, sem qualquer ferramenta externa conectada.

Junto ao módulo, foi desenvolvido um **aplicativo gerenciador** (Jetpack Compose) que permite configurar, direto no aparelho, o IP/porta do proxy e quais apps instalados devem ser interceptados — exatamente como a maioria dos módulos Magisk/KernelSU do mercado (ex: seleção de apps por checkbox).

2. O script original

O `flutter+burp.js` é uma técnica pública e amplamente usada na comunidade de pentest mobile (baseada em pesquisa da NVISO sobre o engine do Flutter), que funciona assim:

1. Localiza `libflutter.so` carregada no processo do app.
2. Faz o parsing manual do cabeçalho ELF (segmentos `PT_LOAD` / `PT_GNU_RELRO`) para delimitar as regiões de memória onde procurar.
3. Escaneia a memória procurando as strings `"ssl_client"` e `"Socket_CreateConnect"`, usadas como pontos de referência.
4. A partir dessas strings, localiza (via padrões de instrução `adrp` / `add` no ARM64, ou `lea` relativo a RIP no x64) o endereço real das funções internas `verify_cert_chain` (verificação do certificado) e `GetSockAddr` (montagem do endereço de destino da conexão).
5. Instala 3 hooks: um em `GetSockAddr` para capturar o ponteiro do `sockaddr`; um em `socket()` para sobrescrever IP/porta desse `sockaddr` com os valores do proxy configurado; e um em `verify_cert_chain` para forçar o retorno "certificado válido".

Nenhuma dessas três funções é exportada/pública — por isso o script precisa "descobrir" os endereços em tempo real a cada execução, em vez de simplesmente chamar uma função pelo nome.

2.1 Por que um módulo Zyigisk, e não Frida ou LSPosed

Essa é a pergunta mais importante do projeto, e vale respondê-la antes de qualquer detalhe técnico: se a técnica de bypass já existia como script Frida, por que reescrevê-la como módulo?

Frida: excelente para descobrir, inadequado para operar

O Frida é imbatível na fase de pesquisa — você edita JavaScript, recarrega e vê o efeito em segundos. Foi com ele que a técnica original foi desenvolvida, e continua sendo a ferramenta certa para *descobrir* como um app funciona. O problema aparece quando você precisa *usar* isso num trabalho real:

- **Depende de sessão ativa.** É preciso um `frida-server` rodando no aparelho e, normalmente, um cabo USB e um terminal aberto. Fechou o terminal, perdeu o hook.
- **Não sobrevive a reinício.** Cada boot exige subir o servidor e reinjetar tudo.
- **Precisa capturar o processo no nascimento.** Para hookar algo que acontece na inicialização do app, é preciso usar `-f` (spawn) e não anexar depois — o que na prática significa reabrir o app pelo Frida a cada teste.
- **É ruidoso e facilmente detectável.** O `frida-server` abre uma porta em escuta e deixa rastros conhecidos (nomes de thread, strings em memória, entradas em `/proc`). Apps com proteção anti-tamper detectam isso trivialmente — e detectar Frida é literalmente a primeira coisa que qualquer SDK de proteção mobile faz.
- **Inviabiliza observação de longo prazo.** Comportamento em segundo plano, push notifications, sincronização periódica, um teste de várias horas, ou simplesmente entregar o aparelho configurado para outra pessoa reproduzir — nada disso funciona amarrado a um cabo.

LSPosed/Xposed: camada errada do sistema

Aqui o motivo não é conveniência, é **impossibilidade técnica direta**. O Xposed (e o LSPosed, sua implementação moderna) hooka principalmente **métodos Java/ART** — é a ferramenta certa para interceptar chamadas do SDK do Android, alterar retornos de métodos de uma classe, contornar verificações escritas em Java/Kotlin.

O ponto crítico: **no Flutter, nada disso está em Java**. O engine embute o próprio BoringSSL, e tanto a validação de certificado (`verify_cert_chain`) quanto a montagem do endereço de destino (`GetSockAddr`) são funções **nativas, não exportadas, dentro do `libflutter.so`**. Não existe método Java para hookar. Um módulo LSPosed teria de fazer exatamente o mesmo trabalho nativo que o FlutterTap faz — resolver endereços por desmontagem e aplicar hook inline — só que carregando por cima todo o runtime do Xposed, sem ganhar nada em troca.

Esse é também o motivo pelo qual **instalar o certificado CA do Burp no Android não funciona** com apps Flutter: o app ignora o repositório de certificados do sistema por completo. E, como o Flutter também não respeita o proxy configurado no Wi-Fi, não há atalho de configuração — o tráfego simplesmente sai sem passar pelo interceptador.

O que o módulo Zygisk resolve

	Frida	LSPosed / Xposed	FlutterTap (Zygisk)
Alcança código nativo do BoringSSL	Sim	Não diretamente	Sim
Persiste após reiniciar o aparelho	Não	Sim	Sim
Funciona sem cabo/ferramenta externa	Não	Sim	Sim
Porta em escuta / processo externo	Sim (detectável)	Não	Não
Pega o app desde o primeiro instante	Só com spawn manual	Sim	Sim
Zero impacto em apps não selecionados	N/A	Depende do módulo	Sim (<code>DLCLOSE</code>)
Velocidade de iteração no desenvolvimento	Excelente	Média	Baixa (recompilar + reinstalar)

A última linha é uma desvantagem real e honesta: iterar num módulo nativo é bem mais lento do que editar um `.js` e recarregar. É justamente por isso que a divisão de trabalho faz sentido — **Frida para descobrir, módulo para operar**.

Por que o FlutterTap foi criado

A motivação é prática, vinda de trabalho real de análise de tráfego mobile. Num teste de verdade você precisa que o app se comporte **naturalmente ao longo do tempo**: abrir e fechar várias vezes, receber notificação, sincronizar em segundo plano, ser usado por outra pessoa que não sabe o que é Frida. Precisa também de um ambiente **reproduzível** — capaz de ser reconfigurado por qualquer um tocando em checkboxes num app, sem terminal e sem comando decorado.

O FlutterTap transforma uma técnica que era um procedimento manual e frágil em **infraestrutura**: instala uma vez, escolhe os apps, e funciona sozinho a cada boot. A escolha de apps-alvo por pacote existe justamente para isso — o módulo não é um interceptador global; ele fica inerte em todo app que você não selecionou, se descarregando da memória imediatamente (seção 5), o que reduz tanto o risco de efeito colateral quanto a superfície de detecção.

3. Arquitetura da solução

3.1 Módulo nativo (C++, Zygisk)

Lógica do script original	Equivalente no módulo nativo
<code>findAppId()</code>	<code>AppSpecializeArgs::nice_name</code> , lido em <code>preAppSpecialize</code>
<code>BURP_PROXY_IP</code> / <code>PORT</code> fixos no código	<code>/data/adb/fluttertap/config.json</code> , editável pelo app gerenciador
<code>parseElf()</code>	<code>elf_utils.cpp</code>
<code>scanMemory()</code> (busca por padrão com curingas)	<code>mem_scan.cpp</code>
<code>Instruction.parse</code> (Capstone por trás do Frida)	Capstone, linkado diretamente (<code>addr_resolver.cpp</code>)
Hook em <code>GetSockAddr</code> (onEnter)	<code>DobbyInstrument</code>
Hook em <code>socket()</code> (sobrescreve <code>sockaddr</code>)	<code>DobbyHook</code> na função global da libc
Hook em <code>verify_cert_chain</code> (onLeave)	<code>DobbyHook</code> com wrapper que força o retorno

A parte de resolução de endereço (ARM64 e x64) foi portada **instrução por instrução**, incluindo peculiaridades do script original (explicadas na seção 4), usando a mesma biblioteca de desmontagem que o próprio Frida usa por baixo dos panos (**Capstone**), em vez de decodificar manualmente os opcodes — isso elimina uma classe inteira de bugs sutis que uma reimplementação "à mão" correria o risco de introduzir.

3.2 Sistema de configuração compartilhada

O arquivo `/data/adb/fluttertap/config.json` guarda IP/porta do proxy, se a interceptação está ativa e a lista de pacotes-alvo. Ele fica *fora* da pasta do módulo (`/data/adb/modules/fluttertap`), para sobreviver a atualizações/reinstalações do módulo. Como o processo do zygote pode não ter permissão SELinux para ler arquivos em `/data/adb` diretamente, a leitura é feita através do **processo companion** do Zygisk (que roda como root irrestrito), via um protocolo simples sobre socket Unix.

3.3 App gerenciador (Jetpack Compose)

- Lista os apps instalados no aparelho e permite marcar quais devem ser interceptados (com busca).
- Campos para IP/porta do proxy, com validação, e um interruptor geral de ativação.
- Detecta root e o backend ativo (Magisk/KernelSU/APatch) e se o módulo está instalado — apenas para exibição, já que os três implementam a mesma API Zygisk.
- Grava a configuração via shell root (`libsu`), codificada em base64 para não precisar escapar caracteres especiais.
- 7 idiomas: Português (Brasil), Inglês, Espanhol, Francês, Árabe, Hindi e Chinês, trocáveis direto no app.
- Tema claro/escuro sempre acompanhando o tema do sistema (Compose + `values-night/`), corrigindo o problema relatado de telas com fundo branco mesmo com o celular em modo escuro.

4. Peculiaridades do script preservadas de propósito

Para o módulo se comportar **exatamente** como o script original, algumas particularidades — que poderiam parecer "erros" à primeira vista — foram mantidas intencionalmente:

- **Sem suporte a 32 bits.** O script original só trata `Process.arch == 'arm64'` ou `'x64'` — não existe um caminho para ARM32/x86 32 bits nem no original. Por isso o módulo também só é compilado para `arm64-v8a` (aparelhos reais) e `x86_64` (emuladores).
- **"a última correspondência vale".** O `Memory.scan` do script chama o callback para cada ocorrência encontrada, sempre sobrescrevendo a mesma variável — ou seja, se um padrão aparecer mais de uma vez, vale o *último* encontrado, não o primeiro. A busca de memória do módulo nativo reproduz esse mesmo comportamento.
- **Varredura byte a byte no x64.** Os trechos de "rastrear para trás" no x64 avançam 1 byte por vez tentando decodificar uma instrução válida a cada posição (em vez de avançar pelo tamanho real da instrução anterior). Isso foi copiado exatamente assim, pois é o que foi validado empiricamente contra builds reais do BoringSSL/Flutter.
- **Caso especial do app da Alibaba — deliberadamente NÃO portado.** O script trata separadamente o padrão de bytes do app `com.alibaba.intl.android.apps.poseidon`, que difere dos demais. Esse caso especial existia apenas porque os padrões de bytes originais embutiam registradores específicos; a varredura por mnemônicos adotada aqui (seção 6.5) casa nos dois casos, tornando a exceção desnecessária.

5. Melhorias deliberadas (sem mudar o comportamento observável)

Três dos pontos abaixo (captura por thread, posicionamento do guard de SIGSEGV e resolução do `socket()`) tiveram de ser reimplementados para funcionar sob o Zygisk Next Linker — o que está detalhado na **seção 12**. O comportamento final é o descrito aqui; a seção 12 explica por que a primeira implementação não servia.

O script original foi feito para uma sessão curta e supervisionada por um humano. O módulo roda sozinho, durante toda a vida do app, então alguns pontos foram reforçados sem alterar o que acontece na rede:

- **Captura do sockaddr por thread** (em vez de uma única variável global), evitando condição de corrida entre conexões simultâneas.
- **Proteção contra SIGSEGV/SIGBUS** em volta de toda leitura de memória "heurística" — se um endereço calculado acabar sendo inválido, o módulo registra um aviso e desiste daquele trecho, em vez de derrubar o app do usuário.

- **Ocupação zero em apps não selecionados.** Antes de fazer qualquer trabalho, o módulo verifica se o processo atual está na lista de apps escolhidos; se não estiver, ele se descarrega da memória imediatamente (`DLCLOSE_MODULE_LIBRARY`).
- **Hook do `socket()` via Dobby direto**, e não pela API oficial de PLT hook do Zygisk — porque essa API só é válida até o fim do `postAppSpecialize`, e `libflutter.so` normalmente só carrega bem depois disso, durante a inicialização normal do app.

6. Desafios encontrados durante o desenvolvimento

Esta seção documenta os problemas reais enfrentados ao compilar o projeto pela primeira vez, e como foram resolvidos — útil para quem for dar manutenção no projeto depois.

6.1 Licença restritiva do Zygisk Next

O plano inicial era usar o header da API Zygisk a partir do repositório `ZygiskNext`, mas esse projeto mudou de licença e agora proíbe explicitamente copiar/extrair qualquer trecho de código dele. A solução foi usar a fonte oficial e correta: o repositório `topjohnwu/zygisk-module-sample`, mantido pelo próprio criador do Magisk, cujo header é licenciado em 0BSD (equivalente a domínio público) exatamente para uso livre por desenvolvedores de módulos.

6.2 Repositório Dobby com o backend Linux/Android quebrado

O commit mais recente do Dobby (biblioteca de hooking inline usada para os hooks internos) não compila para Android: há uma inconsistência real no próprio código-fonte deles entre o nome de um campo de struct (`RuntimeModule.base` vs. o uso de `.load_address` em vários arquivos) e entre um campo e um método (`MemRange.start` vs. `start()`). Isso foi confirmado de forma independente: o workflow de CI do próprio projeto Dobby define um job que deveria publicar um pacote pré-compilado para Android, mas esse artefato simplesmente não aparece nos releases — sinal de que o job falha silenciosamente há tempos.

A solução foi fixar o submódulo Git do Dobby em `025e9fc` — um patch local sobre o commit upstream `e9fe7fb` (de abril de 2023), o último antes da reestruturação que introduziu essa inconsistência, e aplicar dois pequenos ajustes locais por cima:

1. Uma função (`Logger::Shared()`) definida em um header estava sem a palavra-chave `inline`, causando erro de "símbolo duplicado" ao linkar.
2. O alvo de biblioteca compartilhada `dobby` (que não é usado pelo projeto — só a versão estática é) foi desativado apenas para Android, já que ele também falha ao linkar pelo mesmo motivo.

6.3 APIs experimentais do Jetpack Compose Material 3

Durante a build do app gerenciador, o componente `ExposedDropDownMenuBox` (usado inicialmente no seletor de idioma) e o `TopAppBar` básico se mostraram marcados como API experimental nesta versão do Compose, quebrando a build por padrão. O seletor de idioma foi reescrito com um `DropDownMenu`

simples (API estável), e foi adicionado um opt-in explícito (`@OptIn(ExperimentalMaterial3Api::class)`) para o restante.

6.4 Ambiente de build

A máquina tinha o Java 26 como padrão do sistema, versão recente demais para o Gradle 8.11.1 rodar. A solução foi apontar o `JAVA_HOME` para o JDK 21 que já vem embutido no próprio Android Studio instalado, evitando precisar baixar/installar outro JDK à parte.

6.5 Compatibilidade com builds mais novos do engine Flutter (app de demonstração)

Para conseguir capturas de tela e evidências sem usar apps de clientes, foi selecionado o **DVFA (Damn Vulnerable Flutter App)** — um app de treinamento open-source (MIT), mapeado nas 10 vulnerabilidades do OWASP Mobile Top 10, com APK pronto na aba Releases do GitHub (`github.com/Schmiemandev/dvfa`). O APK oficial não declarava a permissão `android.permission.INTERNET` (confirmado via `aapt dump badging`), então foi recompilado localmente (decompilado com `apktool` , permissão adicionada, reempacotado e assinado com a chave de debug) só para viabilizar o teste — sem alterar nenhuma outra funcionalidade do app.

Ao testar contra esse app, o módulo falhou em resolver o endereço de `verify_cert_chain` . A investigação revelou **três incompatibilidades reais** entre a lógica original do script e como esse binário específico (compilado com uma toolchain mais nova) é estruturado — nenhuma delas é um bug do FlutterTap em si, e sim uma limitação da técnica original que também afetaria o script Frida:

1. **Biblioteca mapeada direto do APK.** Builds recentes do Android/AGP usam `extractNativeLibs="false"` por padrão, então a `libflutter.so` não vira um arquivo próprio — fica mapeada por offset dentro do `base.apk` . A busca original (por caminho de arquivo em `/proc/self/maps`) não encontra nada nesse caso. Corrigido trocando para `dl_iterate_phdr` , que o bionic reporta corretamente nos dois casos.
2. **Segmentos de rodata e texto combinados.** Esse binário específico funde os segmentos ELF de rodata e de código executável em um único `PT_LOAD` (comportamento padrão de versões mais novas do linker lld), enquanto a lógica original assume que são dois segmentos separados. Corrigido identificando "o segmento executável" pela flag `PF_X` em vez de assumir qual seria o segundo `PT_LOAD` .
3. **Padrão de bytes preso a um registrador específico.** O padrão de bytes original do script para achar o par de instruções `adrp / add` (e o equivalente x64 com `lea`) na verdade fixa, nos "nibbles fixos" do padrão, o registrador específico que o compilador escolheu no binário onde o script foi originalmente desenvolvido. Um binário compilado com registradores diferentes para o mesmo cálculo simplesmente não bate com o padrão.

Corrigido buscando pela instrução em si (via Capstone) e validando o endereço calculado, independente de qual registrador foi usado — o que também tornou desnecessário o caso especial do app da Alibaba que o script original tinha (esse caso especial existia só por causa dessa mesma fragilidade).

Após as três correções, o módulo resolveu os endereços corretamente e instalou os hooks com sucesso no DVFA também.

6.6 Limitação conhecida: resolução do GetSockAddr em um build específico

Mesmo depois das três correções acima, o DVFA continuou sem interceptar o tráfego real de rede: os endereços "resolvem" e os hooks são instalados normalmente, mas a reescrita do `sockaddr` nunca dispara nas requisições reais do app (confirmado inclusive forçando uma conexão totalmente nova, alternando o Wi-Fi antes de tentar de novo — descartando a hipótese de reaproveitamento de uma conexão já existente).

Enquanto isso, o mecanismo já havia funcionado de ponta a ponta, com tráfego real capturado, contra um app de produção real de um cliente (testado de forma independente por Eduardo, no próprio celular — seção 10.4) — evidência de que o problema é específico desse build do DVFA, não da lógica do módulo em si.

A explicação mais provável: a resolução do `GetSockAddr` depende de contar quantas instruções `b1` existem a partir do `Socket_CreateConnect` e pegar o alvo da 2ª chamada (mesma lógica do script original). Nesse build específico do DVFA, esse mesmo procedimento provavelmente aponta para uma função diferente, que *parece* uma função de preenchimento de `sockaddr` (mesmo formato de prólogo, escreve a família do endereço no offset 0, zera um buffer do tamanho de um `sockaddr_storage`) mas não é de fato a função chamada nas conexões TCP reais desse app.

Essa investigação não foi levada adiante: seria necessário reverter todo o grafo de chamadas do `Socket_CreateConnect` desse build específico, sem garantia de solução rápida, e esse app de teste não é crítico para o projeto — a decisão (após conversa) foi não arriscar mexer na lógica de hook compartilhada por causa de um único app de teste.

Encaminhamento: o DVFA foi então descartado como app de demonstração e substituído por dois outros apps de treinamento públicos, ambos confirmados funcionando de ponta a ponta sem nenhuma mudança de código adicional — ver seção 10.

7. Estrutura do projeto

```
Modulo SukiSu/
├─ module/                                Módulo Zygisk nativo (Gradle + CMake)
│   └─ src/main/cpp/
│       ├── main.cpp                      Ponto de entrada Zygisk (pre/postAppSpecialize)
│       ├── module_config.*              Schema e matching de pacotes-alvo
│       ├── companion.*                  Processo companion (leitura root do config)
│       ├── elf_utils.*                  Parsing do ELF de libflutter.so
│       ├── mem_scan.*                   Busca de padrões de bytes com curinga
│       ├── addr_resolver.*              Resolução de endereços via Capstone
│       ├── hooks.*                      Instalação dos hooks via Dobby
│       └─ third_party/                  Capstone e Dobby (submódulos git)
│   └─ template/                         module.prop, customize.sh, uninstall.sh, action.sh
├─ manager-app/                          App gerenciador (Kotlin + Jetpack Compose)
├─ scripts/build_module_zip.sh            Empacota o .zip flashável final
└─ docs/                                 Esta documentação
```

8. Compatibilidade

Item	Suporte
Versões do Android	Android 10 (API 29) até Android 17 (API 37, mais recente disponível no momento)
Arquiteturas	arm64-v8a (aparelhos reais) e x86_64 (emuladores)
Root	Magisk (Zygisk nativo), KernelSU / SukiSu Ultra (com Zygisk Next ou NeoZygisk), APatch
Zygisk Next Linker	Compatível (ver seção 12). Validado com o recurso ativo no Zygisk Next 1.4.3.
Ambientes validados em hardware	OnePlus 5 / Android 10 / Kitsune Magisk + NeoZygisk 2.3 • Pixel 8a / Android 17 / SukiSu Ultra + Zygisk Next 1.4.3 (com ZN Linker)
Idiomas do app	Português (Brasil), Inglês, Espanhol, Francês, Árabe, Hindi, Chinês

9. Aviso legal e ético

Esta ferramenta contorna a verificação de certificado TLS de apps Flutter para fins de **análise de tráfego** (pentest mobile, engenharia reversa, pesquisa de segurança). Ela deve ser usada apenas

em aplicativos que você tem autorização para testar (seus próprios apps, ou em um engajamento de teste de segurança autorizado). O módulo não tem nenhum alvo específico embutido — a seleção de quais apps interceptar é sempre manual, feita pelo usuário através do app gerenciador.

10. Validação em dispositivo real

O módulo foi validado em **dois aparelhos físicos**, deliberadamente diferentes entre si para cobrir extremos de versão de Android e de implementação de root:

- **OnePlus 5, Android 10, arm64-v8a** — root via Magisk (fork "Kitsune Magisk" v27.2) com Zygisk fornecido pelo módulo **NeoZygisk** (esse fork não usa o Zygisk nativo).
- **Pixel 8a, Android 17, arm64-v8a** — root via **SukiSu Ultra** (fork do KernelSU) com **Zygisk Next 1.4.3**, incluindo o recurso **Zygisk Next Linker** ativo (ver seção 12).

O Burp Suite rodava na máquina do desenvolvedor, escutando em `192.168.15.13:8083`, na mesma rede Wi-Fi dos aparelhos.

10.1 Apps de demonstração escolhidos

Para ter evidências reais sem usar nenhum app de cliente nas capturas, foram usados dois apps de treinamento de segurança **públicos e oficiais**, ambos em Flutter:

App	Autor / licença	Por que serve como demonstração
Ostorlab Insecure Android App <code>github.com/Ostorlab/ostorlab_insecure_android_app</code>	Ostorlab (empresa de segurança mobile), Apache-2.0	Módulo Flutter com um teste dedicado de tráfego TLS (<code>TLSTraffic</code>), fazendo uma requisição HTTPS real para <code>ostorlab.co</code>
VulnApp <code>github.com/anasachoury/VulnApp</code>	anasachoury	App Flutter simples mapeado no OWASP Mobile Top 10, com uma chamada de rede real (HTTP, para <code>httpbin.org</code>)

Nenhum dos dois tinha um APK pronto (nem em Releases, nem em artefatos de CI) — foi necessário instalar o Flutter SDK e compilar os dois a partir do código-fonte. Ambos também estavam com a permissão `android.permission.INTERNET` faltando no manifesto (o mesmo tipo de descuido encontrado antes no DVFA e no `burptestapp` — aparentemente comum nesse tipo de app de treinamento pequeno) e precisaram de pequenos ajustes de dependências desatualizadas (ver seção

10.5) para compilar com o toolchain atual. Nenhuma mudança foi feita na lógica de vulnerabilidade/teste dos apps em si.

10.2 Passos executados

1. Empacotamento do módulo (`scripts/build_module_zip.sh`) e instalação via `magisk --install-module`.
2. Compilação dos dois apps de demonstração a partir do código-fonte (Flutter SDK instalado localmente) e instalação via `adb install`.
3. Escrita do `config.json` apontando para o proxy real e para o pacote de cada app, um de cada vez.
4. Reinicialização do aparelho para o Zygisk carregar o módulo.
5. Abertura de cada app, interação com a funcionalidade que dispara a chamada de rede, e acompanhamento do log do módulo (`adb logcat`) e do histórico de proxy do Burp.

10.3 Resultado

Sucesso confirmado com tráfego real nos dois apps, sem nenhuma mudança de código entre eles. O log do módulo mostrou a sequência completa esperada em ambos: `libflutter.so` carregada → endereços de `verify_cert_chain` e `GetSockAddr` resolvidos corretamente via Capstone → hooks instalados → `sockaddr` reescrito a cada nova conexão para `192.168.15.13:8083`.

No **Ostorlab Insecure App**, ao acionar o teste `TLSTraffic`, o Burp capturou e decodificou de verdade uma requisição HTTPS para `https://ostorlab.co/` (resposta HTTP/2 200 OK completa, com o HTML da página) — **sem precisar instalar nenhum certificado no aparelho**, exatamente o comportamento esperado do bypass de verificação de certificado.

No **VulnApp**, ao tocar em "Call Insecure API" na tela de perfil, o Burp capturou a requisição HTTP para `http://httpbin.org/get` — confirmando que o redirecionamento de tráfego funciona independente de a conexão ser HTTPS (com bypass de certificado) ou HTTP simples.

Observação sobre o login do VulnApp: ao testar, não apareceu nenhuma requisição de rede no momento do login. Investigando o código-fonte (`lib/screens/login_screen.dart`), o motivo é que a autenticação é inteiramente local — usuário/senha (`admin / 1234`) estão fixos no próprio código Dart, e a única "espera" é um `Future.delayed` simulando um carregamento, sem nenhuma chamada de rede real. A única requisição de rede de todo o app é mesmo a do botão "Call Insecure API". Não é uma falha do módulo — simplesmente não existe requisição nenhuma para interceptar nesse ponto.

10.4 Validação independente contra um app real de produção

Em uma rodada de testes separada, Eduardo confirmou o módulo funcionando contra um app real de produção de um cliente seu, usando apenas o app gerenciador para selecionar o alvo e configurar o proxy, sem nenhum ajuste de código. Essa validação (feita de forma independente, em outro momento, por outra pessoa) é uma prova mais forte do que qualquer app de teste: confirma que a abordagem genérica (mesma lógica de scan/resolução para qualquer app Flutter, nada hardcoded por aplicativo) funciona em produção.

Foi observado um comportamento onde a interceptação só passou a funcionar depois de reabrir o app gerenciador uma segunda vez. A explicação mais provável não é um bug no gerenciador (que só grava o `config.json`; ele não tem nenhuma ligação em tempo real com o processo nativo) e sim o comportamento normal do Zygisk: hooks só entram em processos *novos*. Se o app-alvo já estava rodando em segundo plano (comum em apps de loja/apps que o usuário mantém aberto), reabri-lo apenas traz o processo existente para frente, sem novo hook — é necessário forçar a parada do app-alvo (não do gerenciador) antes de reabrir para o hook valer, algo que já consta no guia de instalação (seção 11).

10.5 Compilando os apps de demonstração a partir do código-fonte

Como nenhum dos dois tinha APK pronto, foi necessário instalar o Flutter SDK e resolver algumas incompatibilidades de dependências antigas (ambos os projetos têm alguns anos e não foram atualizados para o toolchain atual). Nenhuma dessas mudanças afeta a funcionalidade de rede/TLS testada.

Passo	Detalhe
1. Instalar o Flutter SDK	Baixar o canal stable em <code>flutter.dev</code> , extrair em um caminho curto (ex: <code>C:\fdev\flutter</code>) — caminhos longos/aninhados no Windows causam erro de "nome de arquivo muito grande" durante o <code>pub get</code> .
2. <code>flutter config --android-sdk "<caminho do SDK>"</code>	Aponta o Flutter para o Android SDK já instalado (o mesmo usado pelo módulo nativo).
3. Adicionar <code>android.permission.INTERNET</code>	Faltava no manifesto dos dois apps — sem essa permissão nenhuma conexão de rede é possível, com ou sem o FlutterTap.
4. Remover dependências legadas incompatíveis	No Ostorlab: <code>flutter_android</code> (sem null-safety), <code>flutter_inappwebview/webview_flutter</code> (API de Proguard descontinuada), <code>local_auth</code> e <code>path_provider</code> (versões novas dependem do pacote <code>jni</code> , incompatível com a versão do Kotlin Gradle Plugin atual), e <code>android_intent_plus</code> (exige <code>compileSdk</code>

	mais novo que o do projeto). Todas essas dependências eram usadas só por testes de vulnerabilidade <i>não relacionados</i> a rede/TLS (biometria, webview, intents, etc.) — removidas junto com o código Dart que as usava, sem afetar o teste de TLS que interessa aqui.
5. <code>flutter build apk --debug</code>	Gera o APK em <code>build/app/outputs/flutter-apk/</code> (app normal) ou <code>build/host/outputs/apk/debug/</code> (caso de um "Flutter module", como o do Ostorlab, que gera um host de teste automaticamente).

10.6 Sobre o app de teste descartado (DVFA)

O **DVFA (Damn Vulnerable Flutter App)** foi o primeiro app de treinamento usado para tentar gerar evidências sem apps de cliente. Como documentado na seção 6.5/6.6, testar contra ele foi o que revelou as três incompatibilidades reais de builds mais novos do engine Flutter (e as correções aplicadas), mas o DVFA em si acabou sendo descartado como app de demonstração por causa de uma limitação específica daquele build (resolução do `GetSockAddr` para uma função aparentemente errada). Foi substituído pelos dois apps das seções 10.1–10.3, ambos confirmados funcionando sem ressalvas.

11. Guia de instalação manual (em um aparelho novo)

11.1 Arquivos necessários

Arquivo	Como gerar	O que é
FlutterTap- <versão>.zip	<code>./scripts/build_module_zip.sh</code>	Módulo flashável (Magisk/KernelSU/APatch)
manager-app-debug.apk (ou release)	<code>./gradlew :manager-app:assembleDebug</code>	App gerenciador

Esses dois arquivos são gerados a partir do código-fonte e não ficam versionados no Git — para distribuir prontos, o caminho padrão é anexá-los a uma *Release* do GitHub.

11.2 Passo a passo

1. **Ter root com Zygisk ativo.** Magisk, KernelSU ou APatch instalado; Zygisk ligado em Configurações → Zygisk (no Magisk) ou, se o Zygisk nativo não funcionar bem no seu fork/ROM, instalar e habilitar o módulo **Zygisk Next / NeoZygisk** antes de tudo. Reiniciar depois.
2. **Copiar o .zip do módulo para o aparelho.**
3. **Instalar o módulo** pelo app do Magisk/KernelSU/APatch → aba Módulos → "Instalar a partir do armazenamento" → selecionar o .zip. (Ou, via terminal root: `magisk --install-module /caminho/FlutterTap.zip`.)
4. **Reiniciar o aparelho** — obrigatório, o Zygisk só injeta em processos criados depois de carregado no boot.
5. **Instalar o app gerenciador** (.apk), permitindo "fontes desconhecidas" se necessário.
6. **Abrir o FlutterTap e conceder root** quando solicitado.
7. **Selecionar o(s) app(s)-alvo** e configurar IP/porta do proxy (o IP da máquina rodando o Burp/proxy, na mesma rede Wi-Fi do aparelho).
8. **Forçar parada e reabrir o app-alvo** — o hook só entra em processos novos.

11.3 Configuração manual via `config.json` (sem o app gerenciador)

O app gerenciador é apenas uma interface para um único arquivo JSON. Quem preferir automatizar (script de provisionamento, CI de testes, aparelho sem tela) pode escrever esse arquivo diretamente e obter exatamente o mesmo resultado — o módulo não sabe nem se importa com quem escreveu.

```
/data/adb/fluttertap/config.json
```

O caminho fica **deliberadamente fora** do diretório do módulo (`$MODPATH`). Duas consequências práticas: a configuração **sobrevive a atualizações e reinstalações** do módulo, e o `uninstall.sh` **não apaga** esse arquivo de propósito — quem desinstalar e reinstalar recupera o IP/porta e a lista de apps exatamente como estavam.

```
{
  "enabled": true,
  "proxy_ip": "192.168.15.13",
  "proxy_port": 8083,
  "target_packages": ["com.exemplo.app", "com.outro.app"]
}
```

Chave	Default se ausente	Observação
<code>enabled</code>	<code>true</code>	Interruptor geral: <code>false</code> desliga a interceptação sem apagar a lista de apps.
<code>proxy_ip</code>	<code>203.0.113.1</code>	RFC 5737 (TEST-NET-3), reservado para documentação. Escolhido justamente por <i>não poder</i> colidir com um endereço real, tornando óbvio que ainda não foi configurado.
<code>proxy_port</code>	<code>8083</code>	Mesmo valor do script original.
<code>target_packages</code>	<code>[]</code>	Lista vazia significa nada é interceptado .

Como a correspondência é feita

A comparação é contra o **nome do processo** (o `nice_name` do Zygisk), não estritamente o nome do pacote, e segue duas regras:

1. **Igualdade exata** com uma entrada da lista.
2. **Subprocessos nomeados do mesmo app**: listar `com.exemplo.app` também captura `com.exemplo.app:remote`, `:isolate` e afins.

A segunda regra é relevante na prática porque parte dos apps Flutter executa o trabalho de rede em um subprocesso nomeado — não é preciso listá-los separadamente.

Quando o arquivo é lido

O processo *companion* (root) lê o arquivo **do disco, do zero, a cada especialização de processo** — não existe cache em nenhuma camada. Portanto **não é necessário reiniciar o aparelho nem reinstalar o módulo** após uma alteração:

1. editar o `config.json`;
2. `am force-stop` no app-alvo;
3. reabrir o app.

O `force-stop` não é opcional: o hook só é aplicado a processos **recém-criados**, de modo que um app já em execução continua sem interceptação.

Editando na prática

O arquivo está sob `/data/adb`, então exige root. Permissões esperadas: diretório `755`, arquivo `644`, dono `root`.

```
# Opção A – pipe + tee, sem arquivo intermediário
adb shell "echo '{"enabled":true,\"proxy_ip\":\"192.168.1.10\", \"proxy_port\":8080, \"target_packages\":[\"com.exemplo.app\"]}' | su -c 'tee /data/adb/fluttertap/config.json'"
adb shell su -c "am force-stop com.exemplo.app"

# Opção B – escrever local e enviar (melhor para JSON formatado/legível)
adb push config.json /data/local/tmp/config.json
adb shell su -c "cp /data/local/tmp/config.json /data/adb/fluttertap/config.json"
adb shell su -c "chmod 644 /data/adb/fluttertap/config.json"
```

Armadilha de shell: `adb shell su -c "cat > /data/adb/..."` falha com *Permission denied*, porque o redirecionamento `>` é executado pelo shell **externo** (sem root), e não pelo `su`. Use `tee` ou `cp`, como nos exemplos acima.

Dois modos de falha silenciosa

1. JSON inválido não gera erro visível. Se o parse falhar, o módulo assume os valores padrão por inteiro — e o padrão de `target_packages` é lista vazia. O resultado é que uma vírgula sobrando faz o módulo simplesmente não interceptar nada, sem nenhuma mensagem. Valide o JSON antes de concluir que algo está quebrado.

2. Nome do pacote não é o nome do app. Convém consultar em vez de supor:

```
adb shell pm list packages | grep -i ostorlab
# co.ostorlab.ostorlab_insecure_flutter_app.host    <- note o sufixo .host
```

Para confirmar que a configuração foi aceita, acompanhe o log do módulo:

```
adb logcat -s FlutterTap:V
```

O esperado é `selected for hooking` seguido de `hooks: installed for ... -> proxy IP:PORTA`. Se **nada** aparecer, a causa é uma destas três: o nome não corresponde ao processo, o JSON é inválido, ou o app já estava em execução quando a configuração mudou.

12. Compatibilidade com o Zygisk Next Linker (guia para autores de módulos)

O FlutterTap era, entre os módulos usados no aparelho de teste, o único que conflitava com o recurso **Zygisk Next Linker** — com o recurso ativo, o aparelho subia e ficava com a tela preta. O diagnóstico revelou **dois bugs reais no próprio módulo** (não incompatibilidades de formato do binário), ambos latentes e inofensivos sob o Zygisk tradicional, mas fatais sob esse loader. Como as duas armadilhas são genéricas e podem afetar qualquer módulo Zygisk, esta seção documenta o diagnóstico completo para reaproveitamento por outros desenvolvedores.

Ambiente onde tudo abaixo foi verificado: Pixel 8a, Android 17, SukiSu Ultra (fork do KernelSU) com **Zygisk Next 1.4.3** e a opção Zygisk Next Linker ativa. Após as correções, o módulo funciona com o linker ligado, com captura de tráfego real confirmada no proxy.

12.1 A diferença que causa tudo

O Zygisk Next Linker substitui o linker do sistema por um **loader ELF próprio e minimalista**, que pré-carrega os módulos diretamente no `zygote` durante o boot. Ele está **ativo por padrão desde a versão 1.3.3**, ou seja, não é mais um recurso opcional de nicho.

A diferença crucial de ciclo de vida:

- **Zygisk tradicional (linker do sistema):** a `.so` do módulo é carregada com `dlopen` em cada processo de app, depois do `fork`. O `onLoad()` roda no processo do app.
- **Zygisk Next Linker:** a `.so` é carregada uma vez, no `zygote`. O `onLoad()` roda **no zygote**, e o que ele fizer é **herdado por todo app que nascer dali**.

Praticamente todo problema desta seção decorre dessa mudança: código que era isolado em um único processo passa a afetar o sistema inteiro.

12.2 Armadilha 1: handler de sinal global + DLCLOSE (a mais grave)

Este foi o bug que travava o aparelho. O módulo instalava um handler de `SIGSEGV / SIGBUS` no `onLoad()` (para proteger varreduras de memória), **antes** de saber se aquele processo era alvo. Processos não-alvo então chamavam `DLCLOSE_MODULE_LIBRARY` para se descarregarem — mas o handler **continuava registrado no kernel, apontando para uma biblioteca que acabara de ser desmapeada**.

Por que é apenas latente no Zygisk tradicional e catastrófico sob o ZN Linker: no primeiro caso o ponteiro pendurado fica confinado àquele app e só mata o processo se ele receber um sinal. No segundo, o `onLoad()` roda no zygote, e **disposições de sinal sobrevivem ao `fork()`** — então *todo* app nasce com o handler pendurado. E o ART levanta `SIGSEGV` benigno como mecanismo normal de operação (verificação implícita de ponteiro nulo, página de guarda de stack overflow), de modo que todo app morre no primeiro deles. Inclusive o SystemUI: daí a tela preta.

```
// ERRADO – handler global instalado antes de saber se o processo é alvo
void onLoad(zygisk::Api *api, JNIEnv *env) override {
    install_crash_guard();           // sob ZN Linker, isto roda no zygote
}
void preAppSpecialize(AppSpecializeArgs *args) override {
    if (!isTarget(...))
        api->setOption(zygisk::DLCLOSE_MODULE_LIBRARY);
    // a biblioteca é desmapeada, mas o handler segue registrado apontando para ela
}
```

```
// CORRETO – instalar só em processo-alvo confirmado, onde o módulo permanece carregado
void onLoad(zygisk::Api *api, JNIEnv *env) override {
    api_ = api; env_ = env;         // nada invasivo aqui
}
void postAppSpecialize(const AppSpecializeArgs *args) override {
    if (!shouldHook_) return;       // não-alvos já se descarregaram
    install_crash_guard();           // agora é seguro
}
```

No FlutterTap a instalação fica um passo adiante ainda: dentro de `runHookPipeline()`, já na thread de monitoramento que espera o `libflutter.so` carregar. O princípio é o mesmo — só se instala em processo-alvo confirmado, no qual o módulo permanece carregado.

Regra geral: nada que altere estado global do processo — handlers de sinal, hooks inline em `libc`, `atexit`, `threads` — deve ser feito no `onLoad()`. Sob o ZN Linker isso é efetivamente "fazer no zygote, para todos os apps do sistema". Se o módulo pode se descarregar via `DLCLOSE_MODULE_LIBRARY`, então **qualquer** ponteiro que o processo guarde para dentro dele (handler de sinal, callback registrado, thread em execução) tem de ser removido antes.

12.3 Armadilha 2: `dlsym(RTLD_DEFAULT, ...)` retorna `null`

Corrigido o travamento, o módulo carregava e os hooks no `libflutter.so` instalavam, mas o redirecionamento de tráfego parava de funcionar. O log apontou um único ponto de falha: a resolução de `socket()` da libc via `dlsym(RTLD_DEFAULT, "socket")` retornava `null`.

Motivo: com `RTLD_DEFAULT` (e igualmente `RTLD_NEXT`), o bionic determina o escopo de busca a partir da **biblioteca chamadora** — ele mapeia o endereço de retorno de volta para um `soinfo` do linker. Como a `.so` foi mapeada pelo loader próprio do ZN Linker, o linker do sistema **não tem registro algum dela**, essa busca falha e `dlsym` devolve `null`. Um handle explícito não depende dessa etapa.

```
// ERRADO – retorna null quando o módulo foi mapeado pelo ZN Linker
void *addr = dlsym(RTLD_DEFAULT, "socket");
```

```
// CORRETO – handle explícito dispensa a resolução pela biblioteca chamadora
void *addr = nullptr;
if (void *libc = dlopen("libc.so", RTLD_NOW | RTLD_NOLOAD)) { // libc já está sempre mapeada
    addr = dlsym(libc, "socket");
    dlclose(libc);
}
if (addr == nullptr) // fallback: Zygisk tradicional
    addr = dlsym(RTLD_DEFAULT, "socket");
```

Vale notar o contraste: `dl_iterate_phdr` **funciona** normalmente — o Zygisk Next o redireciona para uma implementação interna própria que enxerga corretamente as bibliotecas do app (o módulo usa isso para localizar o `libflutter.so`, inclusive em APKs com `extractNativeLibs=false`). O problema é específico das formas de `dlsym` que dependem do chamador.

12.4 Falso culpado: relocations e symbol versioning

Esta subseção existe para poupar tempo de quem for investigar o mesmo problema. A página "Zygisk Next Linker" da wiki lista quatro "Unimplemented features", e a leitura ingênua dessa lista custou um diagnóstico errado e um patch inútil neste projeto.

Os três primeiros itens da lista estão **tachados** (`~~texto~~` no markdown), cada um com link para a versão que os implementou. O tachado aparece no HTML renderizado, mas **desaparece silenciosamente** em qualquer conversão de markdown para texto puro — restando uma lista que parece dizer que nada disso é suportado. Situação real:

Recurso ELF	Situação	Desde
Relocation de TLS	Suportado	1.4.0

Android packed relocation (<code>DT_ANDROID_REL/RELA</code>)	Suportado	1.3.0-RC4
RELR relative relocation (<code>DT_RELR</code>)	Suportado	1.3.0-RC4
GNU Symbol Versioning (<code>DT_VERNEED</code>)	Não implementado	—

E mesmo o versionamento de símbolos, na prática, **não é o que quebra módulos**: toda `.so` compilada com NDK moderno carrega entradas `VERNEED` contra `libc.so / libdl.so` (visíveis como sufixos `@LIBC` nos símbolos importados), e ainda assim o ZN Linker vem ativo por padrão carregando a maioria dos módulos — logo ele tolera essas entradas, resolvendo símbolos por nome e ignorando a versão. O bionic não tem, na prática, múltiplas versões do mesmo símbolo que exigissem discriminação.

Conclusão prática: não gaste tempo removendo `VERNEED` com `patchelf --clear-version-requirement` ou LIEF, nem desativando RELR/packed relocations por flag de link. Nada disso era o problema aqui. Procure primeiro por **estado global do processo** (seção 12.2) e por **dependência do chamador em APIs `d1*`** (seção 12.3).

Como reforço opcional (defesa em profundidade, não correção), este projeto também trocou os três `thread_local` que possuía por *thread-specific data* do pthread (`pthread_key_create / pthread_getspecific / pthread_setspecific`). Um `thread_local` gera relocations de TLS dinâmico (`R_AARCH64_TLSDESC` , `R_X86_64_DTPOD64`) — suportadas desde a 1.4.0, mas o caminho de TLS ainda recebeu correções na 1.4.1. As funções do pthread são chamadas comuns de libc, resolvidas por relocations triviais que qualquer loader trata de forma idêntica. O `scripts/build_module_zip.sh` passou a falhar o build se alguma relocation de TLS dinâmico reaparecer no binário.

12.5 Como diagnosticar (ferramentas que realmente ajudaram)

Ferramenta / sinal	Para que serve
<code>ksud module list</code> (campo <code>description</code> do <code>zygisk-su</code>)	Mostra o estado do Zygisk Next em tempo real, incl. o flag <code>ZL</code> (linker ativo). Desde a 1.4.1 falhas de carregamento de módulo aparecem aqui e na WebUI — confira isso antes de teorizar.
<code>zygiskd linker <system builtin></code> em <code>/data/adb/modules/zygisk-su/bin/</code>	Alterna entre o linker do sistema e o ZN Linker em tempo real, sem reinstalar — ideal para provar que um bug é específico de um dos dois loaders.
<code>/data/tombstones/</code>	Relatórios nativos de crash. O campo <code>Process uptime</code> diz <i>quando</i> o processo morreu, o que ajuda a separar "falhou ao carregar no boot" de "morreu em uso".

Log do módulo via <code>logcat -s SuaTag:V</code>	Foi o que localizou a Armadilha 2 com precisão: todo o pipeline aparecia funcionando e uma única linha de erro apontava o ponto exato. Vale logar qual caminho de resolução funcionou , não só que falhou.
<code>llvm-readelf -d / -r / --dyn-syms</code>	Inventário do binário: tags dinâmicas, tipos de relocation e símbolos importados. Útil para <i>descartar</i> hipóteses de formato ELF rapidamente.
Zygisk Next 1.4.3+: atribuição de crash por módulo e exportação de bugreport (<code>/data/local/tmp/zygisknext-bugreport-*/</code>)	A própria Zygisk Next passa a apontar qual módulo causou um crash do sistema.

Duas armadilhas de diagnóstico encontradas na prática:

- 1. O `dmesg` é praticamente inútil aqui.** O buffer circular do kernel (1024 KB) dá a volta em poucos minutos de uptime — os logs do carregamento de módulos no boot já foram sobrescritos quando alguém vai olhar. Use tombstones e o log do próprio módulo.
- 2. Ausência de crash em um teste curto não prova nada.** Um "conserto" chegou a ser declarado funcional após alguns minutos de aparelho ligado e ocioso; o travamento real só apareceu cerca de 40 minutos depois, quando apps passaram a ser abertos de fato. Esta classe de bug só se manifesta quando o *caminho de falha* executa — no caso, quando o ART levanta um `SIGSEGV` rotineiro. Exercite o caminho real (abrir, trocar e reabrir apps) antes de considerar corrigido.

Por fim, um detalhe de infraestrutura de teste que economiza tempo: um módulo protetor de bootloop (como o *Yet Another Bootloop Protector*) funciona como excelente rede de segurança nesses testes, porque desativa o módulo culpado sozinho e devolve o aparelho utilizável em vez de deixá-lo em tela preta. Mantenha o **seu** módulo *fora* da allowlist dele enquanto testa — é justamente ele que você quer que seja desativado automaticamente se algo der errado.

13. Próximos passos

✓ Testar o módulo em dispositivo real contra apps Flutter e validar a captura no Burp — **concluído** em dois aparelhos (seção 10), incluindo confirmação independente contra um app real de produção (seção 10.4).

✓ Ajustar a resolução de endereços para builds mais novos do engine Flutter/BoringSSL — **três correções aplicadas e validadas** (seção 6.5); um caso específico (`GetSockAddr` em um build particular, seção 6.6) ficou documentado como limitação conhecida.

✓ Compatibilidade com o **Zygisk Next Linker** — **concluída e validada com o recurso ativo** (seção 12), com três bugs reais corrigidos no próprio módulo.

✓ Capturas de tela do app gerenciador e dos dois apps de demonstração — **feitas e versionadas** em `docs/screenshots/`.

✓ Licenciamento — **MIT** (`LICENSE`), com `THIRD_PARTY.md` e os textos das licenças de Dobby (Apache-2.0) e Capstone (BSD-3) embarcados no próprio `.zip`, como exigem essas licenças para redistribuição binária.

6. **Resolver o pin do submódulo Dobby** — hoje ele aponta para um commit que existe apenas na máquina de desenvolvimento, então `git clone --recurse-submodules` não funciona para terceiros. O patch está preservado em `patches/dobby-android-build.patch`; a solução pretendida é publicar um fork do Dobby com esse commit e reapontar o `.gitmodules`. *Bloqueia a publicação.*
7. Escrever o README completo do GitHub (tutorial de instalação passo a passo com as capturas já disponíveis).
8. Preparar e enviar o módulo para a página oficial de módulos do Magisk ([Magisk-Modules-Repo](#)), seguindo os requisitos de submissão desse repositório.

— Eduardo Lopes